

Experiment No.	5
Experiment Title	Decision Tree and Random Forest: A Comparative Classification Study
Student Name	Sayed Afras S
Register Number	3122247001059
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	14.08.2026

1 Objective

- Implement a Decision Tree classifier.
- Extend it into a Random Forest ensemble and see how much of a difference bagging actually makes.
- Study how hyperparameters (mainly tree depth) affect overfitting vs generalization.
- Pick the best hyperparameters using proper 5-Fold Cross-Validation instead of just eyeballing the test score.
- Compare the single tree against the ensemble on the same test set and understand *why* one wins over the other.

2 Theory

Decision Tree Classifier

A Decision Tree recursively splits the feature space based on impurity measures, at each node it picks the feature (and threshold) that best separates the classes, and keeps splitting until some stopping condition is hit (max depth, minimum samples, or pure leaf nodes).

Key Concepts:

- **Gini Index / Entropy** – both measure how "mixed" the classes are at a node, a pure node (all one class) has 0 impurity. Gini is a bit faster to compute, Entropy is more theoretically grounded (comes from information theory), in practice they usually give pretty similar trees.
- **Tree depth and node splitting** – deeper trees can carve out more complex decision boundaries, but also start fitting noise in the training data past a certain point.
- **Overfitting in deep trees** – a fully grown tree (no depth limit) can basically memorize the training set, getting near 100% training accuracy while doing noticeably worse on unseen data. This is the single tree's biggest weakness.

Limitation: Decision Trees have high variance, a small change in the training data can lead to a very different tree structure.

Random Forest Classifier

Random Forest is an ensemble of many Decision Trees, each trained on a different bootstrapped sample of the data (sampling with replacement), and each split only considers a random subset of features rather than all of them.

Key Ideas:

- **Bootstrap Aggregation (Bagging)** – each tree sees a slightly different version of the training data, so the trees end up different from each other.
- **Random Feature Selection** – at every split, only a random subset of features (controlled by `max_features`) is considered, this decorrelates the trees further so they don't all make the same mistakes.
- **Majority Voting** – the final prediction is just whatever class most of the individual trees voted for (or the averaged probability, for `predict_proba`).

Advantage: Averaging over many decorrelated trees cancels out a lot of the individual variance, so Random Forest usually generalizes better than any single tree in it.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC), <code>wdbc.data</code>
Dataset Source	UCI Machine Learning Repository – https://archive.ics.uci.edu
Number of Samples	569
Number of Features	30 numerical attributes (mean, standard error and "worst" value of 10 base measurements like radius, texture, perimeter, etc.)
Number of Classes	2 – Benign (0) and Malignant (1)
Class Balance	357 Benign (62.7%), 212 Malignant (37.3%)
Missing Values	0
Duplicate Rows	0
Train-Test Split	80% Train (455 rows), 20% Test (114 rows), stratified, <code>random_state=42</code>

This dataset is refreshingly clean compared to the others used so far in this lab, zero missing values and zero duplicates, so preprocessing here was really just the train-test split, no imputation or cleanup needed. The `id` column was dropped right after loading since its just a row identifier with no predictive value (this is one dataset where nobody made the mistake of one-hot encoding the ID column).

4 Exploratory Data Analysis

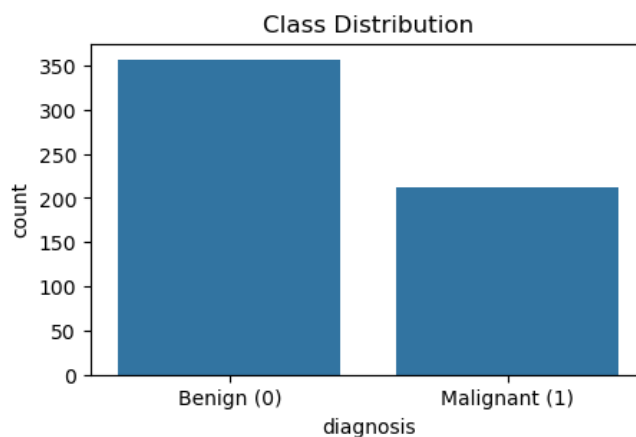


Figure 1: Class distribution – Benign vs Malignant

Inference: The dataset is moderately imbalanced (roughly 63/37), not extreme, but its still worth checking precision/recall separately rather than just relying on accuracy, especially since missing a malignant case (a false negative) is a much more serious mistake in this context than a false positive.

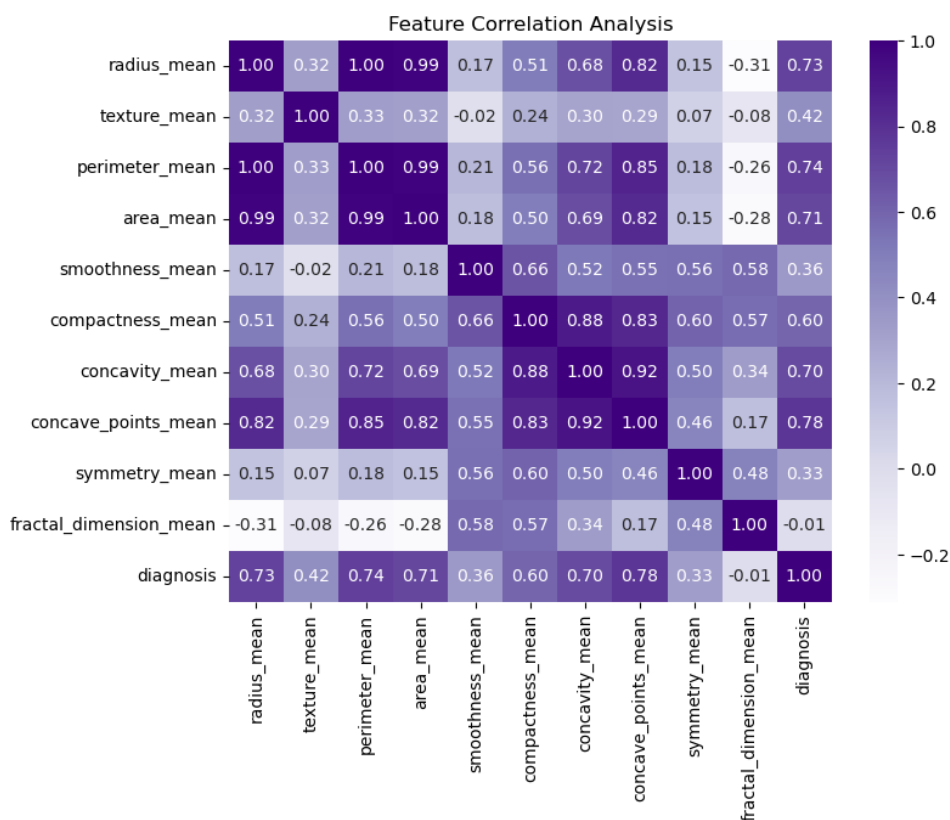


Figure 2: Correlation heatmap of the mean-value features

Inference: `radius_mean` and `perimeter_mean` are correlated at a near-perfect 1.00, and `area_mean` sits right behind at 0.99 with both, which makes total geometric sense

since perimeter and area are basically derived from radius for a roughly circular shape. `concave_points_mean` has the strongest correlation with the diagnosis itself (0.78), followed by `perimeter_mean` (0.74) and `radius_mean` (0.73), so tumor shape/concavity looks like a stronger signal than just size alone. `fractal_dimension_mean` barely correlates with diagnosis at all (-0.01), so on its own it is close to useless as a predictor, though tree models can still combine it with other features in ways a simple correlation number wouldn't capture.

5 Implementation Details

Decision Tree – Hyperparameter Search

Listing 1: Decision Tree search space

```

1 dt_param_grid = {
2     'criterion': ['gini', 'entropy'],
3     'max_depth': [3, 5, 7, 10, None],
4     'min_samples_split': [2, 5, 10],
5     'min_samples_leaf': [1, 2, 4],
6 }
7 dt_grid = GridSearchCV(DecisionTreeClassifier(random_state=42),
8                        dt_param_grid, cv=5,
9                        scoring=['accuracy', 'f1'], refit='
                        accuracy')
```

This is a $2 \times 5 \times 3 \times 3 = 90$ combination grid, each evaluated with 5-fold CV.

Random Forest – Hyperparameter Search

Listing 2: Random Forest search space

```

1 rf_param_grid = {
2     'n_estimators': [50, 100, 200],
3     'max_depth': [3, 5, 10, None],
4     'max_features': ['sqrt', 'log2'],
5     'bootstrap': [True],
6 }
7 rf_grid = GridSearchCV(RandomForestClassifier(random_state=42),
8                        rf_param_grid, cv=5,
9                        scoring=['accuracy', 'f1'], refit='
                        accuracy')
```

This one is a $3 \times 4 \times 2 \times 1 = 24$ combination grid.

6 Hyperparameter Tuning Results

Table 1: Decision Tree Hyperparameter Evaluation (5-Fold CV, Top 5)

Criterion	Max Depth	Min Samples Split	Min Samples Leaf	Avg CV Acc. (%)	Avg CV F1 (%)
entropy	None	5	2	94.7253	92.9724
entropy	10	5	2	94.7253	92.9724
entropy	7	5	2	94.7253	92.9724
entropy	7	5	1	94.5055	92.7352
entropy	10	5	1	94.5055	92.7352

Note: the top three rows are an exact tie (94.7253% accuracy), all using `entropy` with `min_samples_split=5` and `min_samples_leaf=2`, just at three different depths (7, 10 and unlimited). This actually makes sense, past a depth of 7 or so, extra depth just wasn't needed, the tree naturally stopped growing further once the leaves got pure enough given `min_samples_leaf=2`. The notebook only prints this top-5 table rather than a single `best_params_` line, but whichever of these three tied combinations `GridSearchCV` internally picked, it performs identically on cross-validation, so it does not really change the story.

Table 2: Random Forest Hyperparameter Evaluation (5-Fold CV, Top 5)

n_estimators	Max Depth	Max Features	Avg CV Acc. (%)	Avg CV F1 (%)
50	None	sqrt	96.7033	95.6130
50	10	sqrt	96.7033	95.6130
100	10	sqrt	96.2637	95.0094
100	None	sqrt	96.2637	95.0094
50	None	log2	96.0440	94.6027

Note: Interestingly `n_estimators=50` beats 100 and 200 here (only the top 5 are shown but 200 does not even make the cut), more trees is not automatically better once the forest has already stabilized, and `sqrt` consistently beats `log2` for `max_features` on this dataset (30 features, so `sqrt` $\approx$ 5-6 features per split vs `log2` $\approx$ 5 as well, close but `sqrt` edges ahead here).

Comparing the two best-tuned models: Random Forest's CV accuracy (96.70%) beats Decision Tree's (94.73%) by exactly 2 percentage points, and the same gap shows up in F1 (95.61% vs 92.97%), a first sign that the ensemble is doing its job.

7 5-Fold Cross-Validation Performance Comparison

Table 3: 5-Fold CV Accuracy Comparison (on the Training Set)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Decision Tree	90.11	96.70	92.31	91.21	94.51	92.97
Random Forest	95.60	98.90	93.41	94.51	97.80	96.04

Inference: Random Forest wins in every single fold, not just on average. The gap is smallest in Fold 3 (93.41 vs 92.31, about 1 point) and largest in Fold 1 (95.60 vs 90.11, over 5 points), so Decision Tree's performance swings around a lot more fold to fold (its range is 90.11 to 96.70, almost a 6.6 point spread) while Random Forest stays tighter (93.41 to 98.90, about 5.5 point spread, slightly less swingy and centred higher). This is a direct, visible demonstration of the variance-reduction that bagging is supposed to give you.

8 Test Set Evaluation

Metric	Decision Tree	Random Forest
Accuracy	0.9561	0.9737
Precision	1.0000	1.0000
Recall	0.8810	0.9286
F1-Score	0.9367	0.9630

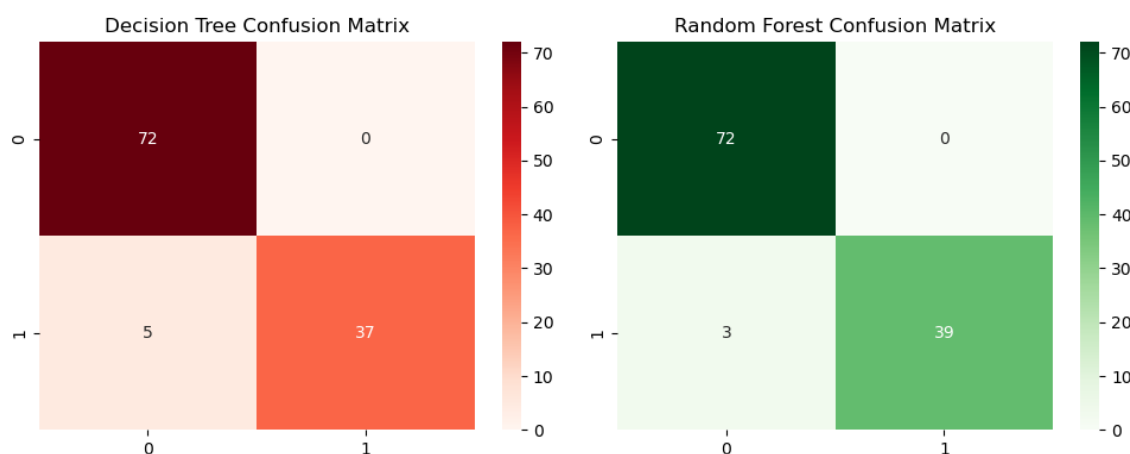


Figure 3: Confusion matrices on the test set (0 = Benign, 1 = Malignant)

Inference: Both models get every single Benign prediction right (72/72, zero false positives, matching the perfect 1.0 precision for both). The real difference shows up on the Malignant side: Decision Tree misses 5 out of 42 actual malignant cases (recall 0.881), Random Forest misses only 3 (recall 0.929). In a medical screening context, those missed malignant cases (false negatives) are the ones that actually matter the most, so Random Forest's lower miss-rate here is a genuinely meaningful improvement, not just a cosmetic accuracy bump.

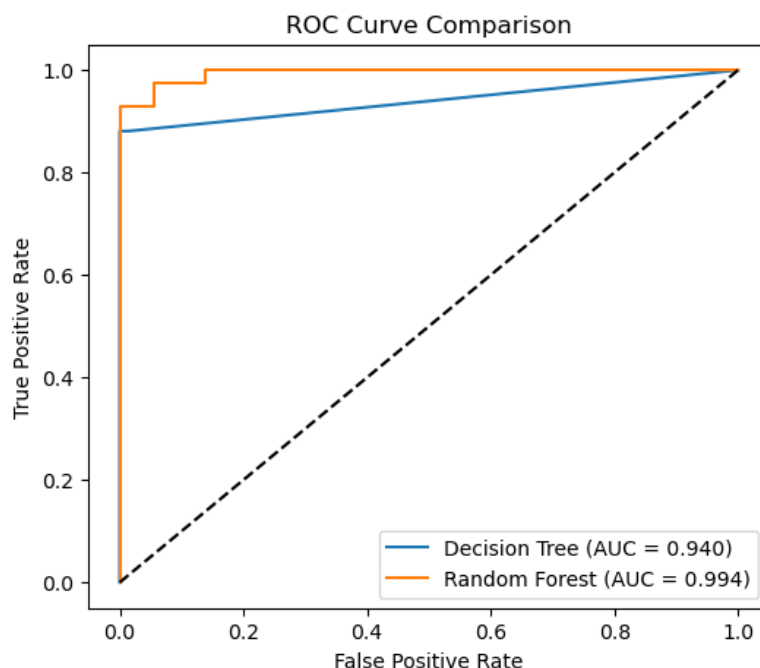


Figure 4: ROC curve comparison

Inference: Random Forest’s ROC curve hugs the top-left corner much tighter than Decision Tree’s, and the AUC backs this up clearly, 0.994 for Random Forest vs 0.940 for Decision Tree. A single tree’s ROC curve is naturally a bit more jagged/staircase-like since its decision boundaries are just a handful of axis-aligned splits, while the forest’s averaged probabilities give a smoother, more confident ranking of malignant vs benign cases.

9 Observation Questions

1. How does tree depth affect overfitting in Decision Trees?

Shallow trees (like `max_depth=3`) tend to underfit, they can’t capture enough of the actual decision boundary and both training and CV accuracy stay lower. As depth increases, training accuracy keeps climbing (a fully unrestricted tree can hit close to 100% on the training data) but CV/test accuracy plateaus and can even start dropping past a point, since the extra depth starts fitting noise specific to the training fold. In this experiment, the best depths (7, 10 or unrestricted, tied at 94.73% CV accuracy) all landed close together, suggesting the tree naturally stopped needing more splits once `min_samples_leaf=2` was satisfied, depth alone was not the only thing controlling complexity here.

2. Which hyperparameter had the greatest impact on performance?

For the Decision Tree, `criterion` mattered more than expected, every single row in the top-5 table uses `entropy`, gini-based trees just did not make the cut here. For Random Forest, `max_features='sqrt'` clearly beat `'log2'` in every top row, and `n_estimators=50` outperformed both 100 and 200, showing that throwing more trees at the problem past a certain point gave no extra benefit (and would just cost more training time for nothing).

3. How does Random Forest improve generalization?

By training many trees on bootstrapped samples with random feature subsets at each

split, and then averaging their votes, Random Forest cancels out a lot of the noise/variance that any individual tree picks up from its particular training sample. This shows up very directly in Table 3, Random Forest’s fold-to-fold accuracy is both higher on average (96.04% vs 92.97%) and doesn’t swing around as wildly as the single tree’s.

4. Did ensemble learning always improve performance? Why or why not?

In this experiment, yes, Random Forest beat Decision Tree on every metric checked, CV accuracy, all 5 individual folds, test accuracy, recall, F1 and AUC. That said, “always” is too strong a word in general, ensembling helps most when the base learner (a single Decision Tree here) has high variance to begin with, which is exactly the known weakness of trees. For a model that already has low variance, bagging wouldn’t buy much and mostly just adds training/prediction cost. It’s also worth remembering that for this particular dataset, precision was already a perfect 1.0 for both models, so the ensemble’s advantage here is really coming entirely from the recall side (catching more of the actual malignant cases), not from some across-the-board magic improvement.

10 Conclusion

Both the Decision Tree and Random Forest classifiers were implemented and properly tuned using 5-fold cross-validation on the WDBC dataset. The tuned Decision Tree (entropy criterion, depth around 7-10, min_samples_split=5, min_samples_leaf=2) reached a solid 95.61% test accuracy, but Random Forest (n_estimators=50, max_depth=10, max_features=sqrt) pushed that further to 97.37%, with a noticeably better recall (0.929 vs 0.881) and a much stronger AUC (0.994 vs 0.940). The 5-fold CV comparison confirms this was not a fluke of one particular split, Random Forest won every single fold and was also more consistent fold-to-fold. Given that missing a malignant tumor is the costlier mistake in a real diagnostic setting, Random Forest is the clearly better choice here, its ensemble structure trades a bit of interpretability (you can’t just eyeball 50 trees the way you can one) for meaningfully better and more stable generalization, exactly what the bagging theory predicts.

11 References

- [1] Scikit-learn Documentation – Decision Trees. [Online]. Available: <https://scikit-learn.org/stable/modules/tree.html>.
- [2] Scikit-learn Documentation – Ensemble Methods (Random Forest). [Online]. Available: <https://scikit-learn.org/stable/modules/ensemble.html>.
- [3] UCI Machine Learning Repository – Breast Cancer Wisconsin (Diagnostic) Dataset. [Online]. Available: <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>.